Single-Vendor E-Commerce Backend

Daily Execution Guide: Day 1 to Day 15

Har din ka kaam, banane wali APIs, request/response aur test cases

FastAPI | PostgreSQL | SQLAlchemy | Pydantic | JWT | Alembic Migrations
Backend Only | Single Admin Store (No Seller Concept) | Auto COD Payment on Delivery | Strict Stock Validation

Contents

Section	Kya hai
1	Is guide ko kaise use karna hai (daily routine, API conventions, Postman setup)
2	Project summary, roles aur access matrix (Single-Vendor: Sirf Admin aur Buyer)
3	Test data (har din ke test isi data par honge)
4	Database summary (11 tables - `seller_orders` table khatam kar di gayi hai)
5	Business rules (Stock quantity validation, Admin order status update & auto payment completion, reviews)
6	Master API list: kaun si API kis din banegi (total 35 endpoints)
7	Day 1 se Day 15: har din ka complete step-by-step plan, code, APIs aur Test Cases
8	Final checklist (15 testing scenarios, 22 tasks) aur end-to-end demo

1. Is Guide ko Kaise Use Karna Hai

Roz ka routine

1. **Subah:** us din ka section parhein: Goal, Steps aur APIs table.
2. **Kaam:** Steps ko order mein karein. Har API banane ke baad foran Swagger (`/docs`) mein test karein.
3. **Test:** Day ke Test Cases table ke har row ko Swagger ya Postman se chalayein aur Expected result se match karein.
4. **Shaam:** End-of-Day checklist tick karein. Agar koi test fail hai to agla din shuru na karein.
5. **Git:** Day ke end par diya hua commit message use kar ke commit karein. Migration ho to wo bhi commit karein.

Rule: Ek din ka kaam complete aur test hone ke baad hi agla din shuru hoga. Database schema sirf Alembic migrations se badlega, `create_all()` kabhi use nahi hoga.

API Conventions (poore project mein same)

Cheez	Rule
Create success	201 Created + naya object
Read / Update / Delete success	200 OK. Delete par `{"message": "... deleted"}` ya object
Validation error	422 (Pydantic/FastAPI automatically)
Business rule fail (Stock / Cart / Status)	400 Bad Request (e.g. empty cart, invalid status change, ya stock se zyada quantity par: `{"detail": "Itni quantity available nahi hai"}`)
Token missing / invalid / expired	401 Unauthorized
Role galat ya doosre ka resource	403 Forbidden
Record nahi mila	404 Not Found
Duplicate (email, category, review)	409 Conflict
Error body format	`{"detail": "message"}`
Auth header	`Authorization: Bearer <token>`
Pagination	Query: `skip` (default 0), `limit` (default 20, max 100)
Money & Time	Numeric(10,2) 2 decimals; UTC datetime (ISO 8601)

Postman Setup (Day 5 se use hoga)

- Collection banayein: **Ecommerce Backend**. Har Day ke liye ek folder: Day 5 - Auth, Day 7 - Categories, etc.
- Environment variables: `base_url = http://localhost:8000` , `buyer_token` , `buyer2_token` , `admin_token` . (Koi seller token nahi hoga).
- Har request ke Authorization tab mein: Bearer Token = `{{buyer_token}}` ya `{{admin_token}}` .
- Login request ke Tests tab mein token auto-save: `pm.environment.set('buyer_token', pm.response.json().access_token)` .
- Swagger mein: upar Authorize button se token paste karein.

2. Project Summary aur Access Matrix

Single-Vendor Store: Hum khud hi store ke malik (Seller) aur Admin hain. Koi alag seller role nahi hai. Admin khud categories aur products (quantity/stock ke sath) post karta hai. Buyers aate hain, cart mein items dalte hain aur COD order place karte hain. Jab order ban jata hai, to Admin khud order par ja kar status change karta hai aur `delivered` karne par payment ka status khud-ba-khud `completed` ho jata hai.

Role	Kaise banta hai	Kya kar sakta hai
Buyer	Public signup (role hamesha `buyer`)	Products browse, cart mein add (stock limit ke andar), address, checkout, apne orders dekhna, delivered product par review dena
Admin (Store Owner)	Sirf seed script (public signup se nahi)	Categories banana, Products post karna (stock quantity ke sath), Images upload karna, Sab orders dekhna, Order ka status change karna (jis se payment auto-complete ho), Order cancel karna, Users manage karna

Access Matrix

Feature	Public	Buyer	Admin (Hum Khud)
Register (Buyer) / Login	Yes	-	Login only (Seed script se banta hai)
View categories / products / reviews	Yes	Yes	Yes
Category create / update / delete	No	No	Yes
Product create (with quantity) / update / deactivate / images	No	No	Yes
Addresses, Cart, Checkout	No	Own only	No
View orders	No	Own only	All orders
Order status update (pending -> processing -> shipped -> delivered)	No	No	Yes (Delivered par payment auto-complete)
Payment details dekhna	No	Own order	All
Cancel order, manage users	No	No	Yes
Post review	No	Verified buyer	No

3. Test Data (har din yehi use karein)

Is data ko Day 5 se Day 14 tak tests mein use kiya jayega, taake expected results fix rahein. Kyunki ab koi alag seller nahi hai, sirf 2 Buyers aur 1 Admin honge.

Naam	Email	Password	Role
Ali Khan	ali@example.com	Ali@12345	buyer (`buyer_token`)
Sara Noor	sara@example.com	Sara@12345	buyer 2 (`buyer2_token`)
Admin (Store Owner)	admin@example.com	Admin@12345	admin (seed script se, `admin_token`)

Product	Posted By	Category	Price	Stock (Quantity)
Samsung Mobile	Admin (Store)	Mobiles	50000.00	10
Headphones	Admin (Store)	Accessories	3000.00	20
Nike Shoes	Admin (Store)	Shoes	8000.00	5

Standard checkout scenario: Ali ke cart mein Mobile x1, Headphones x2, Shoes x1. Total = 50000 + 6000 + 8000 = Rs. 64,000. Ek hi main order banega Rs. 64,000 ka.
Test address (Ali): Ali Khan, 03001234567, House 12 Street 5, Lahore, 54000, Pakistan.

4. Database Summary (11 Tables)

Multi-vendor wali `seller_orders` table khatam kar di gayi hai, aur `products` mein `seller_id` nahi hai.

Table	Columns	Key constraints
`users`	id, name, email, password_hash, role, is_active, created_at	email UNIQUE; role in (`buyer` , `admin`)
`categories`	id, name, description, created_at	name UNIQUE
`products`	id, category_id, name, description, price, stock, is_active, created_at, updated_at	FK category_id; price > 0; stock >= 0; index on category_id
`product_images`	id, product_id, image_url, is_primary, created_at	FK product_id (CASCADE)
`carts`	id, user_id, created_at, updated_at	user_id UNIQUE
`cart_items`	id, cart_id, product_id, quantity, created_at, updated_at	UNIQUE(cart_id, product_id); quantity > 0
`addresses`	id, user_id, full_name, phone, address_line, city, postal_code, country, is_default	FK user_id (CASCADE)
`orders`	id, buyer_id, shipping_address_id, total_amount, status, created_at, updated_at	FK buyer_id, shipping_address_id (RESTRICT)
`order_items`	id, order_id, product_id, product_name, quantity, unit_price, subtotal	FK order_id, product_id (RESTRICT)
`payments`	id, order_id, amount, payment_method, status, paid_at, created_at	order_id UNIQUE; method = COD; status in (`pending` , `completed`)
`reviews`	id, user_id, product_id, rating, comment, created_at	UNIQUE(user_id, product_id); rating CHECK 1..5

5. Business Rules (yaad rakhne wali cheezein)

1. Stock / Quantity Check Rule (Sab se ahem rule)

- Jab Admin koi product post karta hai, to us ki `stock` (quantity) dalta hai.
- Jab Buyer cart mein product add kare (`POST /cart/items`), cart ki quantity update kare (`PATCH /cart/items/{id}`), ya checkout kare (`POST /orders/`), to backend check karega ke `requested_quantity > product.stock` to nahi hai.
- Agar user available stock se zyada quantity select kare, to **400 Bad Request** return hoga aur message aayega: `{"detail": "Itni quantity available nahi hai"}` .

2. Order Status Transitions & Automatic Payment Update (Admin khud karta hai)

Current Status	Allowed Next	Kaun Karega	Payment par kya asar hoga
`pending`	`processing`	Sirf Admin	Payment status `pending` rahega
`processing`	`shipped`	Sirf Admin	Payment status `pending` rahega
`shipped`	`delivered`	Sirf Admin	**Payment status khud-ba-khud `completed` ho jayega aur `paid_at = current UTC time` set ho jayega!**
`pending` / `processing` / `shipped`	`cancelled`	Sirf Admin	Stock wapas add ho jayega; payment `pending` hi rahegi
`delivered` / `cancelled`	koi change nahi	-	400 Bad Request agar dobara status badalne ki koshish ki

3. Stock aur Cancellation

- Checkout par stock kam hota hai (row lock `SELECT ... FOR UPDATE` ke saath). Order cancel hone par stock wapas add hota hai.
- Product delete = soft delete (`is_active = false`). Purane orders ka snapshot (name, price) change nahi hota.
- Delivery charge: config mein `DELIVERY_CHARGE` (default 0). Total = items total + delivery charge.

4. Review Rule

- Sirf buyer, aur sirf us product par jo uske kisi `delivered` order mein ho. Ek buyer ek product par ek review. Rating 1 se 5.

6. Master API List: Kaun Si API Kis Din (Total 35 Endpoints)

Day	Method	Endpoint	Access	Purpose
Day 1	GET	`/health`	Public	App chal raha hai
Day 2	GET	`/health/db`	Public	Database connected hai
Day 5	POST	`/auth/register`	Public	Buyer signup (Sirf buyer)
Day 5	POST	`/auth/login`	Public	Login, JWT milta hai
Day 6	GET	`/auth/me`	Any logged-in	Current user
Day 7	POST	`/categories/`	Admin	Category create
Day 7	GET	`/categories/`	Public	Categories list
Day 7	PUT	`/categories/{id}`	Admin	Category update
Day 7	DELETE	`/categories/{id}`	Admin	Category delete
Day 8	POST	`/products/`	Admin	Product create (quantity/stock ke sath)
Day 8	GET	`/products/`	Public	Active products + filters
Day 8	GET	`/products/{id}`	Public	Product details
Day 8	PUT	`/products/{id}`	Admin	Product / stock update
Day 8	DELETE	`/products/{id}`	Admin	Product deactivate (soft delete)
Day 8	GET	`/admin/products`	Admin	Sab products (inactive bhi)
Day 9	POST	`/products/{id}/images`	Admin	Multiple images upload
Day 9	DELETE	`/product-images/{id}`	Admin	Image delete
Day 10	POST	`/addresses/`	Buyer	Address add
Day 10	GET	`/addresses/`	Buyer	Apne addresses
Day 10	PUT	`/addresses/{id}`	Buyer (own)	Address update
Day 10	DELETE	`/addresses/{id}`	Buyer (own)	Address delete
Day 10	GET	`/cart/`	Buyer	Cart dekhna
Day 10	POST	`/cart/items`	Buyer	Cart mein add (Stock check ke sath)
Day 10	PATCH	`/cart/items/{id}`	Buyer (own)	Quantity update (Stock check ke sath)
Day 10	DELETE	`/cart/items/{id}`	Buyer (own)	Item remove
Day 11	POST	`/orders/`	Buyer	Checkout (Single order + Stock check)

Day	Method	Endpoint	Access	Purpose
Day 12	GET	`/orders/my-orders`	Buyer	Buyer ki order history
Day 12	GET	`/orders/{id}`	Buyer (own) / Admin	Order details + items + payment
Day 12	GET	`/admin/orders`	Admin	Sab orders (filters ke sath)
Day 12	PATCH	`/admin/orders/{id}/status`	Admin	Order status change + Auto Payment Complete
Day 13	GET	`/payments/{id}`	Buyer (own) / Admin	Payment details dekhna
Day 13	PATCH	`/admin/orders/{id}/cancel`	Admin	Order cancel + stock restore
Day 13	GET	`/admin/users`	Admin	Users list
Day 13	PATCH	`/admin/users/{id}/status`	Admin	User activate / deactivate
Day 14	POST	`/products/{id}/reviews`	Buyer (verified)	Review add
Day 14	GET	`/products/{id}/reviews`	Public	Reviews dekhna

DAY 1: Project Setup aur Environment

Goal	Project ka base, virtual environment, dependencies aur pehla chalta hua FastAPI server.
Andaza waqt	2-3 ghante
Pehle kya zaroori	Python 3.11+, PostgreSQL, Git, VS Code install hon.
Aaj ki APIs	1 endpoint (`GET /health`)
Migration	Koi nahi

Steps (is order mein karein)

1. Project folder banayein: `mkdir ecommerce_backend` , phir `cd ecommerce_backend` .
2. Git init: `git init` . Virtual env: `python -m venv venv` , phir activate karein (Windows: `venv\Scripts\activate`).
3. Dependencies install karein (neeche command), phir `pip freeze > requirements.txt` .
4. Folders banayein: `app/core` , `app/models` , `app/schemas` , `app/routers` , `app/services` , `app/repositories` , `app/dependencies` , `tests` , `uploads` , `scripts` . Har package folder mein khali `__init__.py` .
5. `app/main.py` mein FastAPI app aur `GET /health` likhein (neeche code).
6. `.gitignore` banayein: `venv/` , `.env` , `__pycache__/` , `uploads/*` , `*.pyc` .
7. Server chalayein: `uvicorn app.main:app --reload` . Browser mein `/docs` kholein.
8. `README.md` mein project ka naam aur setup ka skeleton likhein. Pehla commit karein.

```
pip install fastapi "uvicorn[standard]" sqlalchemy psycopg2-binary alembic \
pydantic-settings "python-jose[cryptography]" "passlib[bcrypt]" \
python-multipart email-validator pytest httpx

# app/main.py
from fastapi import FastAPI

app = FastAPI(title="Single-Vendor E-Commerce API", version="1.0.0")

@app.get("/health", tags=["Health"])
def health():
    return {"status": "ok"}
```

Aaj ki files

- `app/main.py`, `requirements.txt`, `.gitignore`, `README.md` (skeleton), Khali `__init__.py` files

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /health`	Public	-	`{"status": "ok"}`	-

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D1-1	Server start	`uvicorn app.main:app --reload`	Console mein 'Application startup complete', koi error nahi
D1-2	Health check	`GET /health`	200 + `{"status": "ok"}`
D1-3	Swagger docs	Browser: `localhost:8000/docs`	Swagger UI khulta hai, `/health` nazar aata hai
D1-4	OpenAPI json	Browser: `localhost:8000/openapi.json`	JSON milta hai
D1-5	Git ignore	`git status`	`venv` aur `.env` track nahi ho rahe

End-of-Day Checklist

- [ ] Virtual env active hai & `requirements.txt` bana hai
- [ ] `/health` 200 deta hai & `/docs` khulta hai
- [ ] Folder structure complete hai & Pehla commit ho gaya

```
Git commit: chore: initial FastAPI project setup
```

DAY 2: Configuration aur PostgreSQL Connection

Goal	Settings `.env` se load hon aur app PostgreSQL se connect ho.
Andaza waqt	2 ghante
Pehle kya zaroori	Day 1 complete, PostgreSQL chal raha ho.
Aaj ki APIs	1 endpoint (`GET /health/db`)
Migration	Koi nahi. Yaad rakhein: `create_all()` nahi chalana.

Steps (is order mein karein)

1. PostgreSQL mein database banayein: `CREATE DATABASE ecommerce_db;` (pgAdmin ya psql se).
2. `.env` banayein aur `.env.example` (bina secrets ke) bhi banayein.
3. `app/core/config.py` : Settings class (`pydantic-settings`) jo `.env` parhe.
4. `app/core/database.py` : `engine` , `SessionLocal` , `Base` (DeclarativeBase) aur `get_db()` dependency.
5. `main.py` mein `GET /health/db` likhein jo `SELECT 1` chala kar connection check kare.
6. Test karein, phir galat password daal kar error behavior dekhein, phir sahi karein.

```
# .env
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/ecommerce_db
SECRET_KEY=change-this-to-a-long-random-string
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
UPLOAD_DIR=uploads
MAX_IMAGE_SIZE_MB=5
MAX_IMAGES_PER_PRODUCT=5
DELIVERY_CHARGE=0
```

Aaj ki files

- `app/core/config.py` , `app/core/database.py` , `.env` , `.env.example` , `app/main.py` (health/db add)

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /health/db`	Public	-	`{"database": "connected"}`	500 agar DB down

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D2-1	DB connected	`GET /health/db`	200 + `{"database": "connected"}`
D2-2	Galat password	`.env` mein password galat kar ke restart, `GET /health/db`	Error (500) aata hai, app crash nahi hoti. Phir sahi kar ke dobara 200
D2-3	Settings load	`print(settings.DATABASE_URL)` temporary	Value `.env` wali aati hai (test ke baad print hata dein)
D2-4	Secrets safe	`git status`	`.env` track nahi ho rahi, `.env.example` track ho rahi hai

End-of-Day Checklist

- [ ] `ecommerce_db` bana hai & `/health/db` 200 deta hai
- [ ] `.env` gitignore mein hai & `.env.example` commit hai

```
Git commit: feat: add settings and PostgreSQL connection
```

DAY 3: SQLAlchemy Models aur Relationships (11 Tables)

Goal	Tamam 11 models, enums, constraints aur relationships likhna (Bina Seller Concept ke).
Andaza waqt	4-5 ghante
Pehle kya zaroori	Day 2 complete.
Aaj ki APIs	Koi naya API nahi
Migration	Aaj sirf models. Tables kal (Day 4) migration se banenge.

Steps (is order mein karein)

1. Enums banayein: `UserRole (buyer, admin)` — *seller role nahi hoga*, `OrderStatus (pending, processing, shipped, delivered, cancelled)`, `PaymentMethod (COD)`, `PaymentStatus (pending, completed)`.
2. Models is order mein likhein (dependency ki wajah se): `user`, `category`, `address`, `product`, `product_image`, `cart`, `cart_item`, `order`, `order_item`, `payment`, `review`. (`seller_order` model nahi banega).
3. Har model mein PK, FK, nullable/default, `created_at` / `updated_at` (UTC) define karein. `Product` model mein `seller_id` nahi hoga, sirf `category_id` hoga.

4. Constraints lagayein: `users.email` unique, `categories.name` unique, `carts.user_id` unique, `UniqueConstraint(cart_id, product_id)`, `UniqueConstraint(user_id, product_id)` reviews mein, `payments.order_id` unique, `CheckConstraint(rating 1..5)`, `CheckConstraint(stock >= 0)`, `CheckConstraint(quantity > 0)`, `CheckConstraint(price > 0)`.
5. Delete rules: `product_images` / `cart_items` / `addresses` = CASCADE; `orders` / `order_items` ke FK = RESTRICT (history safe).
6. Indexes: `products(category_id)`, `orders(buyer_id)`, `order_items(order_id)`.
7. `relationship()` with `back_populates` likhein: `Product.images`, `Cart.items`, `Order.items`, `Order.payment`, etc.
8. `app/models/__init__.py` mein sab 11 models import karein (Alembic ko isi ki zaroorat hogi).

Aaj ki files

- `app/models/user.py`, `category.py`, `address.py`, `product.py`, `product_image.py`
- `app/models/cart.py`, `cart_item.py`, `order.py`, `order_item.py`, `payment.py`, `review.py`
- `app/models/enums.py`, `app/models/__init__.py`

Test Cases (Python Shell)

#	Scenario	Kaise test karein	Expected result
D3-1	Import check	`python -c "from app.models import *"`	Koi ImportError nahi
D3-2	Mapper check	`python -c "from sqlalchemy.orm import configure_mappers; import app.models; configure_mappers()"`	Koi relationship error nahi
D3-3	Model count	`len(Base.metadata.tables)`	11 (11 tables)
D3-4	Relationships	Python shell: `Product`, `Order` ke relationship attributes dekhein	`product.images`, `order.items`, `order.payment` maujood hain
D3-5	Constraints	`Base.metadata.tables['products'].constraints` print karein	`Check(stock >= 0)` aur `Check(price > 0)` nazar aate hain

End-of-Day Checklist

- [ ] 11 model files bani hain (Seller order aur seller_id hata diya gaya hai)
- [ ] Sab FKs, relationships, constraints aur indexes lage hain
- [ ] Import aur mapper check pass

> **Git commit: feat: add all 11 SQLAlchemy models and relationships for single-vendor store**

DAY 4: Alembic Setup aur Pehli Migration

Goal	Alembic configure kar ke 11 tables ko migration se create karna.
Andaza waqt	2-3 ghante
Pehle kya zaroori	Day 3 complete (models ready).
Aaj ki APIs	Koi naya API nahi
Migration	Aaj pehli migration: create all tables.

Steps (is order mein karein)

1. `alembic init alembic` chalayein.
2. `alembic.ini` mein `sqlalchemy.url` khali chorein (hum `env.py` se denge).
3. `alembic/env.py` mein: `settings.DATABASE_URL` set karein aur `target_metadata = Base.metadata`, aur `app.models` import karein.
4. Migration generate karein: `alembic revision --autogenerate -m "create all tables"`.
5. Generated file khol kar review karein: 11 tables, enums, constraints, indexes, delete rules sahi hain?
6. Apply karein: `alembic upgrade head`.
7. pgAdmin/psql mein verify karein: `\dt` se 11 tables + `alembic_version`.
8. Downgrade/upgrade test karein, phir database ko head par wapas laayein.

```
alembic init alembic
alembic revision --autogenerate -m "create all tables"
alembic upgrade head
alembic current
alembic downgrade base   # test ke liye
alembic upgrade head     # wapas head par
```

Aaj ki files

- `alembic.ini`, `alembic/env.py`, `alembic/versions/xxxx_create_all_tables.py`

Test Cases (psql / pgAdmin)

#	Scenario	Kaise test karein	Expected result
D4-1	Tables bane	psql: `\dt`	11 tables + `alembic_version`
D4-2	Revision	`alembic current`	Latest revision ID dikhta hai (head)
D4-3	Downgrade	`alembic downgrade base`	Tamam tables delete (sirf `alembic_version` bacha)
D4-4	Upgrade dobara	`alembic upgrade head`	11 tables wapas aa gaye

#	Scenario	Kaise test karein	Expected result
D4-5	FK check	psql: `\d products`	FK to categories, aur CHECK constraints (`stock >= 0`) nazar aate hain
D4-6	Unique check	psql: 2 users same email insert karein	Duplicate key error aata hai

End-of-Day Checklist

- [ ] Migration file review ho gayi & 11 tables database mein hain
- [ ] `upgrade` / `downgrade` dono chalte hain & Migration Git mein commit hai

```
Git commit: feat: add Alembic and initial migration (11 tables)
```

DAY 5: Buyer Registration aur Login (No Seller Signup)

Goal	Sirf Buyer signup, password hashing aur login se JWT token.
Andaza waqt	4 ghante
Pehle kya zaroori	Day 4 complete (users table maujood).
Aaj ki APIs	2 endpoints (`POST /auth/register` , `POST /auth/login`)
Migration	Koi nahi.

Steps (is order mein karein)

1. `core/security.py` : `hash_password()` , `verify_password()` (bcrypt), `create_access_token()` .
2. `schemas/user.py` : `UserCreate (name, email, password)` — **Note:** Public registration mein role field dene ki zaroorat nahi, ya agar dein to sirf `Literal['buyer']` allowed ho. Seller ya Admin signup allowed nahi hai. Backend khud `role = UserRole.buyer` set karega. Password rule: kam az kam 8 chars, upper, lower, digit, special.
3. `UserResponse (id, name, email, role, is_active, created_at)` , `LoginRequest` , `TokenResponse` .
4. `repositories/user_repository.py` : `get_by_email()` , `get_by_id()` , `create()` .
5. `services/auth_service.py` : register (duplicate email = 409, hash, save with role='buyer') aur login (verify, `is_active` check, token).
6. `routers/auth.py` : `POST /auth/register` aur `POST /auth/login` . `main.py` mein router include karein.
7. Buyer ke liye cart: pehli baar cart use hone par automatically banega (`get_or_create_cart`), register par nahi.
8. Postman collection aur environment variables banayein. Test buyers (Ali aur Sara) register karein, login karein aur `buyer_token` , `buyer2_token` save karein.

Aaj ki files

- `core/security.py`, `schemas/user.py`, `repositories/user_repository.py`, `services/auth_service.py`, `routers/auth.py`

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /auth/register`	Public	`{"name": "Ali Khan", "email": "ali@example.com", "password": "Ali@12345"}`	201 `{"id": 1, "name": "Ali Khan", "email": "ali@example.com", "role": "buyer", "is_active": true}`	409 email exists; 422 invalid email/password/role
`POST /auth/login`	Public	`{"email": "ali@example.com", "password": "Ali@12345"}`	200 `{"access_token": "eyJ...", "token_type": "bearer"}`	401 wrong credentials; 403 inactive user

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D5-1	Buyer 1 register	`POST /auth/register`: Ali	201, role automatically `buyer`, `password_hash` response mein nahi
D5-2	Buyer 2 register	`POST /auth/register`: Sara	201, role `buyer`
D5-3	Duplicate email	Ali ko dobara register karein	409 Conflict
D5-4	Admin/Seller role block	Body mein `role = "admin"` ya `"seller"` bhejein	422 Validation Error (ya ignore ho kar sirf `buyer` bane)
D5-5	Invalid email	`email = "abc"`	422
D5-6	Weak password	`password = "12345"`	422
D5-7	Missing field	`name` nahi bheja	422
D5-8	DB check	psql: `SELECT email, role, password_hash FROM users`	`password_hash` bcrypt (`$2b$...`) hai, plain text nahi
D5-9	Login sahi	`POST /auth/login` Ali	200 + `access_token`
D5-10	Login galat password	Ali + galat password	401
D5-11	Login unknown email	`nobody@example.com`	401

End-of-Day Checklist

- [ ] Buyers register ho rahe hain (Seller ya Admin public signup se nahi ban sakte)
- [ ] Password hashed hai & Login token deta hai
- [ ] Postman environment mein `buyer_token` aur `buyer2_token` save hain

> **`Git commit: feat: add buyer registration and login`**

DAY 6: JWT Verification, Role-Based Access aur Admin Provisioning

Goal	Har protected API token verify kare, Buyer/Admin role check ho aur Admin (Store Owner) script se bane.
Andaza waqt	3-4 ghante
Pehle kya zaroori	Day 5 complete.
Aaj ki APIs	1 endpoint (`GET /auth/me`)
Migration	Koi nahi.

Steps (is order mein karein)

1. `create_access_token` mein payload: `sub = user.id` , `role` , `exp` (`ACCESS_TOKEN_EXPIRE_MINUTES`).
2. `dependencies/auth.py` : `OAuth2PasswordBearer(tokenUrl='/auth/login')` , `get_current_user()` (decode, user fetch, `is_active` check, warna 401).
3. Role guards: `require_buyer` aur `require_admin` . Galat role = 403 Forbidden.
4. `GET /auth/me` likhein.
5. `scripts/create_admin.py` : `ADMIN_EMAIL` / `ADMIN_PASSWORD` se Admin (Store Owner) create kare. Public API se admin nahi banega.
6. Admin script chalayein: `python -m scripts.create_admin` . Admin login karein, `admin_token` Postman mein save karein.
7. Temporary test routes (debug) banayein: `/_t/buyer` , `/_t/admin` jo guards check karein. Test ke baad hata dein.

Aaj ki files

- `dependencies/auth.py` , `core/security.py` (update), `routers/auth.py` (`/me` add), `scripts/create_admin.py`

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /auth/me`	Any logged-in	Header: `Bearer token`	200 `{"id": 1, "name": "Ali Khan", "email": "ali@example.com", "role": "buyer"}`	401 token missing/invalid/expired

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D6-1	Token ke baghair	`GET /auth/me` (no Authorization)	401
D6-2	Garbage token	`Bearer abc123`	401
D6-3	Expired token	`ACCESS_TOKEN_EXPIRE_MINUTES=1` kar ke 1 min baad	401
D6-4	Valid buyer token	`GET /auth/me` with `buyer_token`	200, role = `buyer`
D6-5	Admin create & login	`python -m scripts.create_admin`, phir admin login	Login 200, `/auth/me` role = `admin`
D6-6	Buyer on admin route	`GET /_t/admin` with `buyer_token`	403 Forbidden
D6-7	Admin on admin route	`GET /_t/admin` with `admin_token`	200
D6-8	Inactive user	psql: `UPDATE users SET is_active=false WHERE email='sara@example.com'`; phir Sara ka token	401/403, aur login 403. Test ke baad `is_active` true kar dein

End-of-Day Checklist

- [ ] Token verify ho raha hai & Buyer/Admin guards kaam kar rahe hain
- [ ] Admin sirf script se banta hai & `admin_token` Postman mein hai
- [ ] Temporary test routes hata di gayin

> **`Git commit: feat: add JWT auth, role guards and admin seed script`**

DAY 7: Category APIs (Admin Only Write)

Goal	Admin categories manage kare, sab log categories dekh saken.
Andaza waqt	2-3 ghante
Pehle kya zaroori	Day 6 complete (`admin_token` ready).
Aaj ki APIs	4 endpoints
Migration	Koi nahi.

Steps (is order mein karein)

1. `schemas/category.py` : `CategoryCreate (name, description)` , `CategoryUpdate` , `CategoryResponse` .
2. `repositories/category_repository.py` aur `services/category_service.py` : duplicate name check (409), not found (404).
3. Delete rule: agar category mein products hon to delete block (400, `"Category has products"`).
4. `routers/categories.py` : 4 endpoints, create/update/delete par `require_admin` .
5. Default categories add karein: Electronics, Clothing, Shoes, Mobiles, Accessories (Swagger se Admin token ke sath).

Aaj ki files

- `schemas/category.py` , `repositories/category_repository.py` , `services/category_service.py` , `routers/categories.py`

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /categories/`	Admin	`{"name": "Mobiles", "description": "Smartphones"}`	201 `{"id": 4, "name": "Mobiles", "description": "Smartphones"}`	401, 403, 409 duplicate, 422
`GET /categories/`	Public	-	200 `[{"id": 1, "name": "...", "description": "..."}, ...]`	-
`PUT /categories/{id}`	Admin	`{"name": "Mobile Phones"}`	200 updated category	403, 404, 409, 422
`DELETE /categories/{id}`	Admin	-	200 `{"message": "Category deleted"}`	403, 404, 400 has products

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D7-1	Admin create	`POST /categories/` `admin_token` , name Mobiles	201
D7-2	5 categories	Electronics, Clothing, Shoes, Mobiles, Accessories create karein	Sab 201
D7-3	Duplicate	Mobiles dobara create	409
D7-4	Buyer create	`POST /categories/` `buyer_token`	403 Forbidden
D7-5	No token	`POST /categories/` without token	401
D7-6	Public list	`GET /categories/` (no token)	200, 5 categories
D7-7	Update	`PUT /categories/4` `admin_token`	200, name badal gaya
D7-8	Update not found	`PUT /categories/9999`	404
D7-9	Empty name	`name = ""`	422
D7-10	Delete	`DELETE` khali category, `admin_token`	200, phir GET list mein nahi

End-of-Day Checklist

- [ ] 4 category APIs kaam kar rahi hain & Sirf Admin write kar sakta hai
- [ ] 5 default categories maujood hain & Duplicate name block hai

```
Git commit: feat: add category CRUD APIs for admin
```

DAY 8: Product CRUD & Stock Quantity Setup (Admin Only)

Goal	Hum (Admin) khud products post karein aur unki quantity (stock) dalein; buyers active products browse karein.
Andaza waqt	4-5 ghante
Pehle kya zaroori	Day 7 complete (categories maujood).
Aaj ki APIs	6 endpoints
Migration	Koi nahi.

Steps (is order mein karein)

1. `schemas/product.py` : `ProductCreate (category_id, name, description, price > 0, stock >= 0)` , `ProductUpdate` (sab optional, stock سمیت), `ProductResponse (id, category, name, description, price, stock, is_active, images)` .
2. `repositories/product_repository.py` : create, get_by_id, list_active (filters + pagination), update, deactivate.
3. `services/product_service.py` : category exist check (400). Koi seller ownership check nahi kyunki Admin hi sole owner hai.
4. `routers/products.py` : `POST` (`require_admin`), `GET` list/detail (public, sirf active), `PUT` / `DELETE` (`require_admin`).
5. `GET /products/` filters: `category_id` , `search` (name ilike), `min_price` , `max_price` , `skip` , `limit` .
6. `DELETE` = soft delete (`is_active = false`). Product detail mein inactive product public ko 404 de.
7. `routers/admin.py` : `GET /admin/products` (`require_admin` , inactive products bhi dikhayein).
8. Test products banayein (Admin token se): Samsung Mobile (stock=10, price=50000), Headphones (stock=20, price=3000), Nike Shoes (stock=5, price=8000).

Aaj ki files

- `schemas/product.py` , `repositories/product_repository.py` , `services/product_service.py` , `routers/products.py` , `routers/admin.py`

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /products/`	Admin	`{"category_id": 4, "name": "Samsung Mobile", "description": "...", "price": 50000, "stock": 10}`	201 product (stock = 10 ke sath)	403 not admin; 400 bad category; 422
`GET /products/`	Public	Query: `category_id, search, min_price, max_price, skip, limit`	200 list (sirf active, stock quantity ke sath)	422 bad query
`GET /products/{id}`	Public	-	200 product + images + available stock	404 not found/inactive
`PUT /products/{id}`	Admin	`{"price": 48000, "stock": 12}`	200 updated product	403 not admin; 404; 422
`DELETE /products/{id}`	Admin	-	200 product (`is_active = false`)	403 not admin; 404

API	Access	Request	Success response	Errors
`GET /admin/products`	Admin	Query: filters, skip, limit	200 sab products (active + inactive)	403

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D8-1	Admin create product	`POST /products/` `admin_token` (Samsung Mobile, stock 10)	201, stock = 10 save hota hai
D8-2	Baqi test products	Headphones (stock 20), Nike Shoes (stock 5) via `admin_token`	Dono 201
D8-3	Buyer create block	`POST /products/` `buyer_token`	403 Forbidden
D8-4	Bad category	`category_id = 9999`	400
D8-5	Negative price	`price = -5`	422
D8-6	Negative stock	`stock = -1`	422
D8-7	Public list	`GET /products/` (no token)	200, 3 products
D8-8	Filter category	`GET /products/?category_id=4`	Sirf Mobiles
D8-9	Search	`GET /products/?search=nike`	Nike Shoes
D8-10	Price range	`GET /products/?min_price=2000&max_price=9000`	Headphones aur Nike Shoes
D8-11	Pagination	`GET /products/?limit=2&skip=0`	2 products
D8-12	Detail	`GET /products/{id}`	200 details
D8-13	Admin update stock	Admin `PUT` product stock/price	200 (Test ke baad stock wapas 10, 20, 5 rakhein)
D8-14	Buyer update block	Ali `PUT` product	403 Forbidden
D8-15	Deactivate	Admin `DELETE` Headphones	200, `is_active = false`
D8-16	Hidden after delete	`GET /products/` aur `GET /products/{headphones_id}`	List mein nahi, detail 404
D8-17	Admin view all	`GET /admin/products` `admin_token`	200, inactive Headphones bhi nazar aata hai
D8-18	Reactivate	Admin `PUT` Headphones `is_active = true`	Aage ke tests ke liye Headphones active hona chahiye

End-of-Day Checklist

- [ ] 6 product APIs kaam kar rahi hain & Sirf Admin product post/update kar sakta hai
- [ ] 3 test products apni exact stock quantities (10, 20, 5) ke sath active hain & Soft delete verified

```
Git commit: feat: add admin product CRUD with stock quantity and public filters
```

DAY 9: Multiple Product Image Upload (Admin Only)

Goal	Admin product ki multiple images upload aur delete kar sake.
Andaza waqt	3-4 ghante
Pehle kya zaroori	Day 8 complete.
Aaj ki APIs	2 endpoints
Migration	Koi nahi (`product_images` table pehle se maujood hai).

Steps (is order mein karein)

1. `uploads/` folder banayein aur `main.py` mein `StaticFiles` mount karein: `app.mount('/uploads', StaticFiles(directory='uploads'), name='uploads')`.
2. `services/image_service.py`: validate (type jpg/jpeg/png/webp, size <= `MAX_IMAGE_SIZE_MB`, product par max `MAX_IMAGES_PER_PRODUCT`).
3. File ka asal content (magic bytes) bhi check karein, sirf extension par bharosa na karein.
4. Filename `uuid4 + extension` se banayein (original filename use nahi hoga, path traversal se bachao).
5. File `uploads/products/{product_id}/` mein save karein; DB mein sirf URL (e.g. `/uploads/products/1/abc.jpg`).
6. Pehli image automatically `is_primary = true`.
7. `routers/product_images.py`: `POST /products/{id}/images` (`List[UploadFile]`, `require_admin`) aur `DELETE /product-images/{id}` (`require_admin`).
8. Delete par DB row aur disk file dono hatayein; primary delete ho to agli image primary banao.
9. `ProductResponse` mein images list show ho (Day 8 ka schema update).

Aaj ki files

- `services/image_service.py`, `schemas/product_image.py`, `routers/product_images.py`, `main.py` (static mount)

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /products/{id}/images`	Admin	`multipart/form-data: files` (1 ya zyada)	201 `[{"id": 1, "image_url": "/uploads/products/1/a.jpg", "is_primary": true}]`	403 not admin; 404; 400 invalid type/size/limit

API	Access	Request	Success response	Errors
`DELETE /product-images/{id}`	Admin	-	200 `{"message": "Image deleted"}`	403 not admin; 404

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D9-1	Single upload	Admin `POST /products/1/images`, 1 jpg	201, `is_primary = true`
D9-2	Multiple upload	Ek request mein 2 aur images	201, 2 rows, `is_primary = false`
D9-3	URL khulta hai	Browser: `localhost:8000 + image_url`	Image display hoti hai
D9-4	DB mein URL	psql: `SELECT image_url FROM product_images`	Sirf path/URL, file data nahi
D9-5	Invalid type	`.pdf` ya `.exe` upload	400
D9-6	Fake extension	Text file ko `.jpg` rename kar ke upload	400 (content check)
D9-7	Oversize	6 MB image	400
D9-8	Limit	Product par 6th image upload	400
D9-9	Buyer upload block	`buyer_token` se upload karein	403 Forbidden
D9-10	Product detail	`GET /products/1`	`images` list mein sab images
D9-11	Delete image	Admin `DELETE /product-images/{id}`	200, file disk se bhi gayab
D9-12	Delete primary	Primary image delete karein	Agli image primary ban jati hai
D9-13	Buyer delete block	Ali image delete kare	403 Forbidden

End-of-Day Checklist

- [ ] Multi-upload kaam karta hai & Validations pass
- [ ] Sirf Admin upload/delete kar sakta hai & Files `uploads/` mein hain, git mein nahi

```
Git commit: feat: add multiple product image upload for admin
```

DAY 10: Buyer Addresses, Cart aur Strict Stock Check

Goal	Buyer address manage kare aur cart mein products add/update kare (Agar stock se zyada select kare to error message aaye).
Andaza waqt	5-6 ghante
Pehle kya zaroori	Day 8 complete (products maujood).
Aaj ki APIs	8 endpoints (4 address + 4 cart)
Migration	Koi nahi.

Steps (is order mein karein)

1. **Addresses:** `schemas/address.py` (Create, Update, Response), repository, service, `routers/addresses.py`. Har query mein `user_id` filter.
2. Pehla address automatically `is_default = true`. Naya default set ho to baqi ke `is_default = false`.
3. Doosre buyer ka address: 403 (ya 404). Default address delete ho to koi aur default ban jaye.
4. **Cart:** `services/cart_service.py`: `get_or_create_cart(user_id)`.
5. **Stock Validation Check (Main Requirement):**
   - `POST /cart/items`: Product active ho, `quantity >= 1`. Agar product pehle se cart mein hai to `new_qty = existing_qty + request_qty` warna `new_qty = request_qty`.
   - Check lagayein: `if new_qty > product.stock: raise HTTPException(status_code=400, detail="Itni quantity available nahi hai")`.
   - `PATCH /cart/items/{id}`: Ownership check + Stock check: `if request.quantity > product.stock: raise HTTPException(status_code=400, detail="Itni quantity available nahi hai")`.
6. `DELETE /cart/items/{id}`: item remove.
7. `GET /cart/`: items (product name, `unit_price` abhi ki price, quantity, subtotal) aur cart total.
8. Sab endpoints par `require_buyer`. Admin ko 403.

Aaj ki files

- `schemas/address.py`, `repositories/address_repository.py`, `services/address_service.py`, `routers/addresses.py`
- `schemas/cart.py`, `repositories/cart_repository.py`, `services/cart_service.py`, `routers/cart.py`

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /addresses/`	Buyer	`{"full_name": "Ali Khan", "phone": "03001234567", "address_line": "House 12 Street 5", "city": "Lahore", "postal_code": "54000", "country": "Pakistan", "is_default": true}`	201 address	403; 422
`GET /addresses/`	Buyer	-	200 apne addresses	403
`PUT /addresses/{id}`	Buyer own	Address fields	200 updated	403 not owner; 404
`DELETE /addresses/{id}`	Buyer own	-	200 message	403; 404; 400 agar kisi order mein use hua ho
`GET /cart/`	Buyer	-	200 `{"items": [{"id": 1, "product_id": 1, "name": "Samsung Mobile", "unit_price": "50000.00", "quantity": 1, "subtotal": "50000.00"}], "total": "50000.00"}`	403 not buyer
`POST /cart/items`	Buyer	`{"product_id": 1, "quantity": 1}`	201 cart item	400 `{"detail": "Itni quantity available nahi hai"}` / inactive; 404 product; 422
`PATCH /cart/items/{id}`	Buyer own	`{"quantity": 3}`	200 updated item	403; 404; 400 `{"detail": "Itni quantity available nahi hai"}`; 422 qty < 1
`DELETE /cart/items/{id}`	Buyer own	-	200 message	403; 404

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D10-1	Add address	Ali `POST /addresses/`	201, `is_default = true`
D10-2	Second address	Ali doosra address `is_default = true`	201, pehla ab default nahi
D10-3	List own	Ali `GET /addresses/`	Sirf Ali ke 2 addresses
D10-4	Other buyer's address	Sara `GET/PUT` Ali ka address id	403/404
D10-5	Admin address block	Admin `POST /addresses/`	403
D10-6	Empty cart	Ali `GET /cart/`	200, `items = []`, `total = 0`
D10-7	Add to cart	Ali `POST /cart/items` Mobile x1	201
D10-8	Same product again	Mobile x1 dobara add	Quantity = 2 (duplicate row nahi)
D10-9	**Stock se zyada (Add)**	Mobile x11 add karein (stock 10 hai)	**400 Bad Request:** `{"detail": "Itni quantity available nahi hai"}`
D10-10	**Stock se zyada (Update)**	`PATCH` item quantity = 15 (stock 10 hai)	**400 Bad Request:** `{"detail": "Itni quantity available nahi hai"}`
D10-11	Inactive product	Inactive product add karein	400/404
D10-12	Qty 0	`quantity = 0`	422
D10-13	Update qty valid	`PATCH` item quantity = 1	200, subtotal update
D10-14	Cart total	Mobile x1, Headphones x2, Shoes x1 add kar ke `GET /cart/`	`total = 64000.00`
D10-15	Other buyer's item	Sara Ali ke cart item par `PATCH/DELETE`	403
D10-16	Sara ka cart	Sara `GET /cart/`	Khali (Ali ka cart nahi dikhta)
D10-17	Remove item	Ali `DELETE` cart item	200, cart se gayab
D10-18	Cart wapas set	Remove ke baad Day 11 ke liye cart dobara 64000 wala banayein	Total 64000.00

End-of-Day Checklist

- [ ] 8 APIs (4 address + 4 cart) kaam kar rahi hain & Buyer isolation verified
- [ ] Stock se zyada quantity select karne par `"Itni quantity available nahi hai"` message aata hai
- [ ] Cart total 64000 set hai (kal ke liye)

> **`Git commit: feat: add buyer addresses and cart management with strict stock check`**

DAY 11: Checkout aur Single-Store Order Creation

Goal	Cart se direct Order, Order Items aur COD Payment ek hi transaction mein (Bina seller sub-orders ke).
Andaza waqt	5-6 ghante (sab se important din)
Pehle kya zaroori	Day 10 complete, Ali ke cart mein 64000 ka saman.
Aaj ki APIs	1 endpoint (`POST /orders/`)
Migration	Koi nahi.

Steps (is order mein karein)

1. `schemas/order.py` : `CheckoutRequest (shipping_address_id)` , `OrderResponse (id, status, total_amount, address, items[product_id, product_name, quantity, unit_price, subtotal], payment)` .
2. `repositories/order_repository.py` : create order, order items, payment.
3. `services/order_service.py` : `checkout()` neeche diye algorithm ke mutabiq. Poora function ek DB transaction mein.
4. Stock lock: products ko id ascending order mein lock karein (`SELECT ... FOR UPDATE`) taake deadlock na ho.
5. `routers/orders.py` : `POST /orders/` (`require_buyer`), response 201.
6. Rollback test ke liye temporary exception lagayein, test kar ke hata dein.
7. Race test: do requests ek saath ek aakhri stock item par chalayein.

```
# Checkout ka algorithm (transaction ke andar)
POST /orders/ body: { "shipping_address_id": 1 }
with transaction:
    1. user buyer hai (guard) -> warna 403
    2. cart items load karo; items khali? -> 400 "Cart is empty"
    3. address (id, user_id) se nahi mila? -> 400 "Invalid address"
    4. products ko id order mein lock karo (SELECT ... FOR UPDATE)
    5. har item: product.is_active? -> warna 400 "Product is not active"
    6. har item: if item.quantity > product.stock:
           -> 400 "Itni quantity available nahi hai"
    7. total = sum(product.price * item.quantity) + DELIVERY_CHARGE (price DB se,
client se nahi)
    8. Order(buyer_id, shipping_address_id, total_amount=total, status=pending)
    9. har item ke liye OrderItem(order_id, product_id, product_name, unit_price
snapshot, qty, subtotal)
    10. Payment(order_id, amount=total, method=COD, status=pending, paid_at=None)
    11. product.stock -= item.quantity
    12. cart_items delete (cart row rahe)
commit! Koi bhi exception -> rollback (cart intact, stock unchanged)
```

Aaj ki files

- `schemas/order.py`, `repositories/order_repository.py`, `services/order_service.py`, `routers/orders.py`

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /orders/`	Buyer	`{"shipping_address_id": 1}`	201 `{"id": 1, "status": "pending", "total_amount": "64000.00", "items": [{"product_name": "Samsung Mobile", "quantity": 1, "unit_price": "50000.00", "subtotal": "50000.00"}, ...], "payment": {"method": "COD", "status": "pending", "amount": "64000.00", "paid_at": null}}`	403 not buyer; 400 empty cart / bad address / `"Itni quantity available nahi hai"` / inactive; 401

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D11-1	Admin checkout block	Admin `POST /orders/`	403 Forbidden
D11-2	Empty cart	Sara (khali cart) `POST /orders/`	400 `"Cart is empty"`
D11-3	Invalid address	Ali `shipping_address_id = 9999`	400
D11-4	Other buyer's address	Ali, Sara ka address id de	400
D11-5	**Insufficient stock at checkout**	Ali cart mein Shoes x5 dale; Admin DB/API se Shoes ka stock kam kar ke 3 kar de; phir Ali checkout kare	**400 Bad Request:** **`{"detail": "Itni quantity available nahi hai"}`**, kuch change nahi hua (Test ke baad Shoes stock wapas 5 aur cart mein x1 kar dein)
D11-6	Inactive product	Cart ka product deactivate kar ke checkout	400
D11-7	Success checkout	Ali cart: Mobile x1, Headphones x2, Shoes x1; `POST /orders/`	201, `total_amount = 64000.00`, `status = pending`
D11-8	Order items check	psql: `SELECT * FROM order_items`	3 rows, `product_name` aur `unit_price` snapshot saved
D11-9	Payment row check	psql: `SELECT * FROM payments`	1 row: COD, `pending`, amount 64000, `paid_at = NULL`
D11-10	Stock updated	`GET /products/`	Mobile = 9, Headphones = 18, Shoes = 4

#	Scenario	Kaise test karein	Expected result
D11-11	Cart cleared	Ali `GET /cart/`	`items = []` (cart row db mein maujood)
D11-12	Price snapshot	Admin Mobile ki price 45000 kar de; order items dekhein	Purane order mein `unit_price` abhi bhi 50000
D11-13	Client total ignore	Body mein `total_amount = 1` bhejein	Ignore hota hai, total backend ka
D11-14	Rollback test	Temporary exception laga kar checkout	500, lekin orders/payments mein koi row nahi, stock same, cart intact
D11-15	Race condition	Do buyers ek saath last 1 stock item checkout karein	Sirf ek 201, doosra 400 (`"Itni quantity available nahi hai"`); stock kabhi negative nahi

End-of-Day Checklist

- [ ] Checkout ek transaction mein chalta hai & Stock sahi kam hota hai
- [ ] Cart items delete hote hain & Rollback test pass
- [ ] Temporary exception hata di gayi

> **Git commit: feat: add single-vendor checkout with transaction, stock lock and COD payment**

DAY 12: Order Views, Admin Status Update & Auto Payment Complete

Goal	Buyer apna order track kare; Admin sab orders dekhe aur status badle; `delivered` hone par Payment khud-ba-khud `completed` ho jaye.
Andaza waqt	5-6 ghante
Pehle kya zaroori	Day 11 complete (Ali ka order #1 maujood).
Aaj ki APIs	4 endpoints
Migration	Koi nahi.

Steps (is order mein karein)

1. `GET /orders/my-orders` : Buyer ke apne orders (pagination, newest first).
2. `GET /orders/{id}` : Buyer (owner) ya Admin. Doosra buyer = 403.
3. `GET /admin/orders` : Admin ke liye sab orders ki list, filters (`status` , `buyer_id`), pagination.

4. `PATCH /admin/orders/{id}/status` (`require_admin`):
   - Allowed transitions enforce karein: `pending -> processing -> shipped -> delivered`. Invalid jump (e.g. `pending` se seedha `delivered`, ya `delivered` se wapas `processing`) par 400 Bad Request.
   - **Auto Payment Completion Logic:** Jab Admin status ko `delivered` par change kare, to isi DB transaction ke andar us order ki `payment` record ko uthayein aur:
     `payment.status = PaymentStatus.completed`
     `payment.paid_at = datetime.now(timezone.utc)`
     set kar ke commit karein! Admin ko alag se payment confirm karne ki zaroorat nahi padegi.

Aaj ki files

- `services/order_service.py` (extend), `routers/orders.py` (extend), `routers/admin.py` (orders list & status update add), `schemas/order.py` (extend)

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /orders/my-orders`	Buyer	Query: `skip, limit`	200 `[order...]`	403
`GET /orders/{id}`	Buyer own / Admin	-	200 order + items + payment	403; 404
`GET /admin/orders`	Admin	Query: `status, buyer_id, skip, limit`	200 sab orders	403
`PATCH /admin/orders/{id}/status`	Admin	`{"status": "processing"}` (ya `shipped` / `delivered`)	200 updated order (aur agar `delivered` hai to `payment.status = "completed"` + `paid_at` set)	403 not admin; 400 invalid transition; 404

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D12-1	My orders	Ali `GET /orders/my-orders`	200, order #1
D12-2	Order details	Ali `GET /orders/1`	200, 3 items, payment `pending`
D12-3	Other buyer block	Sara `GET /orders/1`	403 Forbidden
D12-4	Admin order detail	Admin `GET /orders/1`	200
D12-5	Admin orders list	Admin `GET /admin/orders`	200, sab orders dikhte hain
D12-6	Filter orders	Admin `GET /admin/orders?status=pending`	Sirf pending orders

#	Scenario	Kaise test karein	Expected result
D12-7	Buyer on admin list	Ali `GET /admin/orders`	403 Forbidden
D12-8	Buyer status update block	Ali `PATCH /admin/orders/1/status`	403 Forbidden
D12-9	Invalid transition skip	Admin order #1 ko `pending` se seedha `delivered` kare	400 Bad Request (Invalid status transition)
D12-10	Admin -> processing	Admin `PATCH /admin/orders/1/status` `{"status": "processing"}`	200; order status = `processing`; payment abhi bhi `pending`
D12-11	Admin -> shipped	Admin `PATCH /admin/orders/1/status` `{"status": "shipped"}`	200; order status = `shipped`; payment abhi bhi `pending`
D12-12	**Admin -> delivered (Auto Payment)**	Admin `PATCH /admin/orders/1/status` `{"status": "delivered"}`	**200; order status = `delivered` AUR `payment.status = "completed"`, `paid_at` mein current UTC timestamp set ho gaya!**
D12-13	DB Payment check	psql: `SELECT status, paid_at FROM payments WHERE order_id=1`	`status = 'completed'`, `paid_at` NOT NULL
D12-14	Delivered ke baad change	Admin delivered order ko wapas `processing` kare	400 Bad Request

End-of-Day Checklist

- [ ] Buyer apne orders dekh sakta hai & Admin sab orders dekh sakta hai
- [ ] Status transitions (`pending -> processing -> shipped -> delivered`) enforce hoti hain
- [ ] Order `delivered` karte hi payment khud-ba-khud `completed` ho jati hai aur `paid_at` set hota hai

```
Git commit: feat: add order views, admin status workflow and auto payment completion on delivery
```

DAY 13: Payment View, Order Cancellation aur User Controls

Goal	Payment details dekhna, Admin order cancel kar sake (stock restore ke sath), aur users ko activate/deactivate kar sake.
Andaza waqt	4-5 ghante
Pehle kya zaroori	Day 12 complete (order #1 delivered & paid). Cancel test ke liye Ali se ek naya Order #2 (Mobile x1) banwayein jo `pending` ho.
Aaj ki APIs	4 endpoints
Migration	Koi nahi.

Steps (is order mein karein)

1. `GET /payments/{id}` : Buyer (apne order ki payment) ya Admin. Doosra buyer = 403.
2. `PATCH /admin/orders/{id}/cancel` (`require_admin`):
   - Agar order `delivered` ya pehle se `cancelled` hai to 400 Bad Request.
   - Order ka status = `cancelled` karein.
   - Order ke tamam items ki quantity wapas unke `products.stock` mein jama (restore) karein.
   - Payment status `pending` hi rahegi.
3. `GET /admin/users` : users list (filter `role` , `is_active` , pagination).
4. `PATCH /admin/users/{id}/status` : `is_active` true/false. Admin khud ko deactivate nahi kar sakta (400).
5. Deactivated user ka login aur purana token dono reject hon (Day 6 ka logic).

Aaj ki files

- `services/payment_service.py` , `schemas/payment.py` , `routers/payments.py` , `routers/admin.py` (cancel & users endpoints add), `services/admin_service.py`

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`GET /payments/{id}`	Buyer own / Admin	-	200 `{"id": 1, "order_id": 1, "amount": "64000.00", "payment_method": "COD", "status": "completed", "paid_at": "2026-11-12T10:30:00Z"}`	403; 404
`PATCH /admin/orders/{id}/cancel`	Admin	-	200 cancelled order	403; 404; 400 delivered / already cancelled
`GET /admin/users`	Admin	Query: `role, is_active, skip, limit`	200 users (`password_hash` nahi)	403
`PATCH /admin/users/{id}/status`	Admin	`{"is_active": false}`	200 updated user	403; 404; 400 self-deactivate

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D13-1	Buyer payment view (Order #1)	Ali `GET /payments/1`	200, status `completed`, `paid_at` maujood hai
D13-2	Other buyer block	Sara `GET /payments/1`	403 Forbidden
D13-3	Admin payment view	Admin `GET /payments/1`	200
D13-4	Cancel pending order (Order #2)	Admin `PATCH /admin/orders/2/cancel`	200, order #2 status = `cancelled`
D13-5	Stock restored check	`GET /products/1` (Mobile)	Order #2 ka stock wapas add ho gaya (Mobile stock wapas 9 ho gaya)
D13-6	Double cancel block	Order #2 ko dobara cancel karein	400 Bad Request
D13-7	Cancel delivered block	Order #1 (delivered) ko cancel karein	400 Bad Request
D13-8	Buyer cancel block	Ali `PATCH /admin/orders/2/cancel`	403 Forbidden
D13-9	Users list	Admin `GET /admin/users`	Sab users, `password_hash` nahi
D13-10	Deactivate user	Admin `PATCH` Sara `is_active = false`	200
D13-11	Deactivated login	Sara login kare	403 Forbidden
D13-12	Reactivate user	Admin `PATCH` Sara `is_active = true`	200, login phir chalta hai
D13-13	Self deactivate block	Admin apni id deactivate kare	400 Bad Request

End-of-Day Checklist

- [ ] Payment details endpoint kaam kar raha hai
- [ ] Cancel par stock wapas restore hota hai & Delivered order cancel nahi ho sakta
- [ ] Admin user management kaam karta hai

```
Git commit: feat: add payment view, order cancellation with stock restore and user management
```

DAY 14: Verified Product Reviews

Goal	Verified buyers (jinka order delivered ho chuka hai) product par rating aur comment de saken.
Andaza waqt	3-4 ghante
Pehle kya zaroori	Day 12 complete (Ali ka delivered order #1 maujood).
Aaj ki APIs	2 endpoints
Migration	Koi nahi (`reviews` table aur constraints Day 4 mein ban chuke hain).

Steps (is order mein karein)

1. `schemas/review.py` : `ReviewCreate (rating 1..5, comment)` , `ReviewResponse (id, user_name, rating, comment, created_at)` .
2. `services/review_service.py` : verified purchase check: buyer ke kisi `Order` mein `status == delivered` ke saath `OrderItem` mein wo product ho (`order_items + orders` join).
3. Ek buyer ek product par ek hi review: duplicate = 409 Conflict.
4. `routers/reviews.py` : `POST /products/{id}/reviews` (`require_buyer`), `GET /products/{id}/reviews` (public, pagination).
5. `GET /products/{id}` (aur list) mein `average_rating` aur `review_count` shamil karein (aggregate query).
6. Product exist/active check (404).

Aaj ki files

- `schemas/review.py` , `repositories/review_repository.py` , `services/review_service.py` , `routers/reviews.py` , `schemas/product.py` (`average_rating` , `review_count` add)

Aaj ki APIs (define)

API	Access	Request	Success response	Errors
`POST /products/{id}/reviews`	Buyer (verified)	`{"rating": 5, "comment": "Great phone"}`	201 review	403 not verified / not buyer; 404; 409 duplicate; 422 rating
`GET /products/{id}/reviews`	Public	Query: `skip, limit`	200 reviews list	404 product

Test Cases (Swagger / Postman)

#	Scenario	Kaise test karein	Expected result
D14-1	Not purchased	Sara `POST` review Mobile par (khareeda nahi)	403 Forbidden
D14-2	Purchased & delivered	Ali `POST` review Mobile par, rating 5	201 Created
D14-3	Not delivered order	Naya order jo abhi `pending` hai, us product par review	403 Forbidden
D14-4	Duplicate review	Ali dobara Mobile par review de	409 Conflict
D14-5	Rating 0	`rating = 0`	422
D14-6	Rating 6	`rating = 6`	422
D14-7	Admin review block	Admin `POST` review	403 Forbidden
D14-8	No token	`POST` review without token	401
D14-9	Public list	`GET /products/1/reviews`	200, Ali ka review
D14-10	Average rating	Ali ke baad 2nd verified buyer rating 3 de	`average_rating = 4.0`, `review_count = 2`
D14-11	Product not found	`POST /products/9999/reviews`	404

End-of-Day Checklist

- [ ] Verified purchase rule kaam karta hai & Duplicate review block hai
- [ ] Average rating product response mein aati hai

> **`Git commit: feat: add verified product reviews`**

DAY 15: Testing, Security, Documentation aur GitHub

Goal	Poore project ko automated tests, security review, README aur GitHub push se final karna.
Andaza waqt	6-8 ghante
Pehle kya zaroori	Day 1 se Day 14 complete.
Aaj ki APIs	Koi naya API nahi
Migration	Final verification: khali database par `alembic upgrade head`, `alembic current`, `alembic history`.

Steps (is order mein karein)

1. `tests/` mein pytest setup: alag test database (`ecommerce_test`), `conftest.py` fixtures.
2. Automated tests likhein (neeche list) jo Day 5 se 14 ke manual test cases ko cover karein.
3. Fresh database par migration verify: naya khali DB bana kar `alembic upgrade head` , 11 tables check.
4. Security review list complete karein (neeche).
5. Global exception handlers aur consistent error format (`{"detail": "..."}`) check karein.
6. Swagger: tags, summaries, example payloads add karein. Postman collection export karein (`postman_collection.json`).
7. `README.md` likhein: features, tech stack, setup, `.env` variables, migration commands, admin creation, run, test, API list, roles.
8. End-to-end demo (Section 8) ek baar poora chalayein.
9. `requirements.txt` freeze, `.env.example` final check, git commit aur GitHub par push.

pytest ki files

File	Kya test hoga
`tests/conftest.py`	Test DB, client, fixtures: buyer/admin tokens, sample products with stock
`tests/test_auth.py`	Buyer register, login, admin role block, `/me` , token errors
`tests/test_rbac.py`	Buyer aur Admin ke liye allowed/forbidden endpoints
`tests/test_categories_products.py`	Admin CRUD, stock quantity, filters, soft delete
`tests/test_images.py`	Valid/invalid upload, limits, admin guard
`tests/test_cart_address.py`	Buyer isolation, `"Itni quantity available nahi hai"` stock limit check, duplicate rows
`tests/test_checkout.py`	Success, stock limit failure, rollback, race condition
`tests/test_orders_payments.py`	Admin status transitions, auto payment completion on `delivered` , cancel + stock restore
`tests/test_reviews.py`	Verified buyer, duplicate, rating

Security Review List

- [ ] `SECRET_KEY` `.env` mein hai, repo mein nahi
- [ ] Password hash bcrypt; response mein `password_hash` kabhi nahi
- [ ] Har protected route par guard (`require_buyer` / `require_admin`)
- [ ] Buyer isolation checks (cart, address, orders) har jagah
- [ ] SQL injection: sirf ORM/parameterized queries
- [ ] Image upload: type, size, magic bytes content, uuid filename
- [ ] Public signup se admin nahi ban sakta

Test Cases (Final Verification)

#	Scenario	Kaise test karein	Expected result
D15-1	Test suite	`pytest -v`	Sab tests pass
D15-2	Fresh DB migration	Naya DB + `alembic upgrade head`	11 tables ban jate hain, koi error nahi
D15-3	Fresh clone	Naye folder mein clone, README ke steps follow	Project chal jata hai
D15-4	Secrets check	`git log / grep SECRET_KEY`	Koi secret commit nahi hua
D15-5	E2E demo	Section 8 ke steps	Sab steps expected result dete hain

End-of-Day Checklist

- [ ] `pytest` sab pass & Migrations fresh DB par chalti hain
- [ ] `README.md` complete & Postman collection export
- [ ] Security list complete & GitHub par push ho gaya

```
Git commit: docs: add tests, README and final documentation
```

8. Final Checklist aur End-to-End Demo

15 Zaroori Testing Scenarios

#	Scenario	Din
1	Public signup se Admin ya Seller account create na ho (Sirf Buyer bane)	Day 5, 6
2	Buyer category ya product create/update na kar sake (Sirf Admin kare)	Day 7, 8
3	Admin checkout ya cart use na kar sake	Day 10, 11
4	Buyer doosre Buyer ka cart, address ya order access na kar sake	Day 10, 12
5	Cart mein available stock se zyada quantity select karne par `"Itni quantity available nahi hai"` aaye	Day 10
6	Empty cart se checkout fail ho	Day 11
7	Checkout ke waqt agar stock kam par jaye to `"Itni quantity available nahi hai"` aaye aur order reject ho	Day 11
8	Invalid address par checkout reject ho	Day 11
9	Successful checkout par single order + COD pending payment create ho aur stock kam ho	Day 11
10	Successful checkout ke baad cart items clear hon	Day 11
11	Database error par checkout transaction rollback ho	Day 11

#	Scenario	Din
12	Buyer order ka status change na kar sake (Sirf Admin kare)	Day 12
13	Admin jab order status ko `delivered` kare to payment khud-ba-khud `completed` ho jaye aur `paid_at` set ho	Day 12
14	Admin jab pending order cancel kare to stock wapas restore ho jaye	Day 13
15	Invalid image upload reject ho aur non-verified buyer review na de sake	Day 9, 14

End-to-End Demo (poora flow ek baar)

1. Admin script se Admin banayein, Admin login kare.
2. Admin 5 categories banaye (Mobiles, Accessories, Shoes, etc.).
3. Admin products post kare aur unki quantity (stock) dale: Samsung Mobile (stock 10), Headphones (stock 20), Nike Shoes (stock 5), aur images upload kare.
4. Ali (buyer) register + login kare.
5. Ali `GET /products/` se products aur unki available quantity dekhe.
6. Ali cart mein Mobile ki 15 quantity dalne ki koshish kare -> Error aaye: `"Itni quantity available nahi hai"`.
7. Ali cart mein Mobile x1, Headphones x2, Shoes x1 add kare (total 64,000).
8. Ali apna shipping address add kare.
9. Ali `POST /orders/` se checkout kare: 201 Created, order status `pending`, payment COD `pending`.
10. Ali ka cart khali ho jaye; products ka stock kam ho jaye (Mobile 9, Headphones 18, Shoes 4).
11. Admin `GET /admin/orders` mein ja kar order dekhe, aur `PATCH /admin/orders/{id}/status` se status ko pehle `processing`, phir `shipped`, aur phir `delivered` kare.
12. Jaise hi Admin `delivered` kare, payment ka status khud-ba-khud `completed` ho jaye aur `paid_at` timestamp set ho jaye.
13. Ali `GET /orders/{id}` aur `GET /payments/{id}` check kare: order `delivered` aur payment `completed` dikhe.
14. Ali Mobile par 5-star review de (verified buyer).

Final Project Checklist (22 Tasks)

#	Task	Din	Done
1	FastAPI project setup	Day 1	[ ]
2	PostgreSQL connection	Day 2	[ ]
3	11 database tables (Single-vendor schema)	Day 3	[ ]
4	Primary and foreign key relationships	Day 3	[ ]
5	Alembic migrations	Day 4	[ ]
6	Buyer registration & Login	Day 5	[ ]
7	JWT authentication & Admin seed script	Day 6	[ ]

#	Task	Din	Done
8	Role-based permissions (Buyer vs Admin)	Day 6	[ ]
9	Admin Categories CRUD	Day 7	[ ]
10	Admin Product CRUD with Stock Quantity	Day 8	[ ]
11	Multiple product image uploads (Admin)	Day 9	[ ]
12	Buyer addresses management	Day 10	[ ]
13	Cart management with strict stock check (`"Itni quantity available nahi hai"`)	Day 10	[ ]
14	Single-vendor transactional checkout & stock deduction	Day 11	[ ]
15	Cart clearing after successful checkout	Day 11	[ ]
16	Buyer order history & Admin order views	Day 12	[ ]
17	Admin order status updates (`pending -> processing -> shipped -> delivered`)	Day 12	[ ]
18	Automatic COD payment completion & `paid_at` on order delivery	Day 12	[ ]
19	Admin order cancellation with stock restoration & User management	Day 13	[ ]
20	Verified product reviews	Day 14	[ ]
21	Automated API testing (pytest)	Day 15	[ ]
22	README, Postman collection and GitHub push	Day 15	[ ]

Final Project Definition

Hum ek backend-only **Single-Vendor E-Commerce Store** develop karenge using FastAPI, PostgreSQL aur SQLAlchemy. Hum (Admin) khud hi store ke sole seller aur administrator hain. Admin products aur unki available stock quantity post karega. Buyer products browse karega aur cart se COD order place karega. Agar Buyer available stock se zyada quantity select karega to backend `"Itni quantity available nahi hai"` ka validation error dega. Order banne ke baad Admin khud order ka status update karega, aur jab Admin status ko `delivered` mark karega to COD payment ka status khud-ba-khud `completed` ho jayega aur `paid_at` timestamp set ho jayega. Checkout ke baad cart items clear ho jate hain, lekin order history hamesha database mein safe rehti hai.